TASK 3: Design the Control Logic

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

1. Searching for a free slot and when the search stops

The search goes floor by floor (outer loop) and slot by slot (inner loop). The first slot that is null is taken, so the search stops at once with return. If every slot has been checked and none is free, the search ends and returns null. Using return instead of break is the clearest way to stop both loops at once.

2. Fee slabs using if-else

The fee rule is: every started hour is charged in full, so hours = (minutes + 59) / 60. A valid monthly pass parks free. An EV adds Rs 12 per charging unit. If-else is used for the pass check, and integer arithmetic gives the started hours. A switch on vehicle type would also work, but the fee is calculated by each vehicle's own method (Task 7), so no switch is needed here.

3. break, continue and return

return gives the result as soon as a free slot is found, so no further slots are checked (no unnecessary work). break would leave only the inner loop, which is why return is used. continue is not needed here, because every slot is either taken or allocated. In release(), the checks return false as soon as a position is invalid, so later checks are skipped.

4. What happens when no suitable slot is free

allocate() returns null. The caller must check for null before using the result, and no ticket is created. This is how the test 'facility full' passes.

5. Pseudocode: allocate a free slot

```
ALGORITHM AllocateSlot(vehicleNo)
  INPUT:  vehicleNo (String)
  OUTPUT: {floor, slot} of the allocated slot, or NULL if full

  FOR floor <- 0 TO number of floors - 1 DO
      FOR slot <- 0 TO slots on floor - 1 DO
          IF slots[floor][slot] is EMPTY THEN
              slots[floor][slot] <- vehicleNo
              RETURN {floor, slot}
          END IF
      END FOR
  END FOR
  RETURN NULL
END ALGORITHM
```

6. Pseudocode: release a slot

```
ALGORITHM ReleaseSlot(floor, slot)
  INPUT:  floor, slot (integers)
  OUTPUT: TRUE if the slot was occupied and is now free, otherwise FALSE

  IF floor < 0 OR floor >= number of floors THEN RETURN FALSE
  IF slot < 0 OR slot >= slots on floor THEN RETURN FALSE
  IF slots[floor][slot] is EMPTY THEN RETURN FALSE
  slots[floor][slot] <- EMPTY
  RETURN TRUE
END ALGORITHM
```

7. Pseudocode: compute the parking fee

```
ALGORITHM ComputeFee(minutes, hasPass, kWh, isEV)
```

```
  INPUT:  minutes, hasPass (TRUE/FALSE), kWh, isEV (TRUE/FALSE)
  OUTPUT: fee in rupees

  IF minutes < 0 THEN STOP with error
  IF hasPass = TRUE THEN
      fee <- 0
  ELSE
      hours <- (minutes + 59) DIV 60
      fee <- hours * ratePerHour
  END IF
  IF isEV = TRUE THEN
      fee <- fee + kWh * 12
  END IF
  RETURN fee
END ALGORITHM
```

8. Java code and output

The completed ParkingFacility.java follows the pseudocode above. Its output is:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

9. Alternative: free-slot count and a stack of free slots

Instead of scanning, keep a count of free slots on each floor and a stack of free slot numbers. allocate() then pops the top of the stack and decrements the floor count, and release() pushes the slot back. The pseudocode becomes:

```
ALGORITHM AllocateWithStack(vehicleNo)
  IF total free count = 0 THEN RETURN NULL
  pos <- POP(freeStack)            // a stored (floor, slot) pair
  slots[pos.floor][pos.slot] <- vehicleNo
  freeCount[pos.floor] <- freeCount[pos.floor] - 1
  RETURN pos
END ALGORITHM

ALGORITHM ReleaseWithStack(floor, slot)
  IF slot is already free THEN RETURN FALSE
  slots[floor][slot] <- EMPTY
  freeCount[floor] <- freeCount[floor] + 1
  PUSH((floor, slot), freeStack)
  RETURN TRUE
END ALGORITHM
```

Approach	allocate() time	release() time	Extra space (F floors, S slots each)
Scan (our code)	O(F x S) worst case, O(1) if the first slot is free	O(1)	O(1) extra
Stack of free slots	O(1)	O(1)	O(F x S) for the stack, plus O(F) counters

The scan is simpler and uses almost no extra memory, but it can take up to F x S steps when the facility is nearly full. The stack makes both operations O(1), at the cost of storing up to F x S positions. For a small campus facility (for example 5 floors of 100 slots, 500 positions), the extra memory is trivial, so the stack is the better choice when allocation speed matters.